Judge/Verifier Redesign — Research Log

Living document tracking the full process of redesigning the evaluation (Verifier + Judge) stage of the Agentic Multi-LLM Validation System, for research/FYP writeup purposes. Updated as work progresses — see the bottom for the most recent entry.

---

1. Starting point

Original pipeline: Verifier (phi3, free-form prose fact-check) → Judge (qwen2.5, 5-criteria rubric: Factual Accuracy, Completeness, Clarity, Relevance, Depth of Reasoning, /50) → winner decision. The Judge read the Verifier's prose as unstructured context but never independently verified its claims — Judge scores were effectively dependent on trusting phi3's narrative at face value.

Known issue discovered earlier: the judge/verifier model pairing had changed mid-dataset at an earlier point in the project (rows before a certain id used an older Mistral-based judge with a ~69% A-win-rate bias; rows after switched to phi3+qwen2.5 with ~43% A-win-rate) — training a downstream replacement model on the mixed dataset taught it to reconcile two disagreeing labeling standards. This directly motivated the decision to regenerate a clean, single-regime dataset from scratch (Section 3).

2. Verifier/Judge redesign — 8-factor dual independent scoring

Goal: reduce the Judge's dependence on the Verifier's unstructured opinion, and get a genuine cross-check between two independent models.

Change: introduced a shared 8-factor rubric (`agents/rubric.py`), used identically by BOTH agents: 1. Factual Correctness 2. Question Relevance 3. Context Relevance (RAG-document alignment, or topical relevance if none) 4. Faithfulness / Groundedness 5. Completeness 6. Hallucination-Free 7. Clarity 8. Consistency

(Replaces the old 5-factor judge-only rubric, /50 → /80.)

- **Verifier (phi3)** now independently scores both answers on all 8 factors (structured, not prose) — it does NOT declare a winner.
- **Judge (qwen2.5)** independently scores the same 8 factors AND declares the winner, informed by (but not copying) the Verifier's scores.
- New parser (`utils/rubric_parser.py`) extracts structured scores from either agent's output with the same logic.
- New signal: **verifier-judge agreement** — `1 - normalized mean absolute difference` between their two independent totals. Validated live: caught a genuine disagreement case (Verifier scored an answer 76 vs 57 where Judge scored both 76/76 — agreement correctly read 0.88, not 1.0).

3. Weighted confidence scoring system

Combines multiple signals into one 0-1 confidence score for the pipeline's winning answer, used to gate a regeneration loop (retry ≤2x if below threshold).

Signals (weights sum to 1.0, missing signals renormalize): - `judge` (0.40) — Judge's rubric total /80 - `similarity` (0.15) — cosine similarity of winning answer to question - `model_agreement` (0.10) — BERTScore between answer_a and answer_b (roberta-large, baseline-rescaled) - `verifier_judge_agreement` (0.10) — see Section 2 - `wikipedia` (0.15) — external fact-check (Wikipedia search + summary API, gated by a relevance check so an unrelated article is never trusted) - `wikidata` (0.10) — second, structurally different encyclopedia source added per mentor request (entity search + description, gated the same way)

Threshold calibrated from data (10th percentile of historical weighted scores) rather than guessed: **0.68**. `MAX_REGENERATION_ATTEMPTS = 2`.

4. Wikidata integration notes

Wikidata's `wbsearchentities` API is a **label matcher** (autocomplete-style over entity names), not full-text search like Wikipedia's — searching the raw question ("capital of France") matched on literal label words and surfaced irrelevant entities. Fixed by extracting candidate proper-noun phrases from the **answer** (which names real entities) and falling back to the cleaned question, trying each against the label search until one clears a relevance gate (cosine sim ≥ 0.35 against the question). Validated: 0.82 for a correct answer, 0.44 for an incorrect one (found "Germany" from the wrong answer's own text), correctly abstains (`None`) when there's no real entity to match (common for the compound/analytical question style used in this project's question bank).

5. Fresh dataset regeneration (v2)

Built `generate_training_data_v2.py`: mirrors the live pipeline exactly (including the regeneration loop), logs **every** attempt (not just the final one) with all raw signal values stored separately — so any weight combination can be recomputed later without regenerating data. Writes to a new, separate database (`data/judge_training_v2.db`) so this dataset can never mix with the old one.

6. Infrastructure issues encountered and fixed during the run

- **Recurring hangs** traced to orphaned `llama-server.exe` subprocesses (children of `ollama.exe serve`) that survive a normal parent-process restart on Windows (no cascade-kill) — they silently accumulated across manual restarts, eating VRAM until the next verifier/judge call wedged. Fixed by having the watchdog explicitly enumerate and kill `llama-server.exe` processes directly, not just the `ollama serve` parent.
- **Orphaned watchdog shells**: `TaskStop` on the monitoring harness did not reliably kill the underlying shell process in this environment — multiple watchdog instances accumulated and raced each other, causing duplicate generator launches. Fixed by killing shells directly by PID going forward.
- **Auto-recovery** built into `watchdog_v2.sh`: if no question completes within a stall threshold, it automatically kills the generator, kills all orphaned `llama-server.exe`, restarts Ollama's serve process, and relaunches the generator — the full manual recovery sequence, now self-healing (validated working end-to-end, ~20s to full recovery).
- **VRAM over-subscription root cause**: running 2 questions concurrently each needing up to 4 different ~6-9GB models was the underlying driver of the recurring stalls. Fixed by reducing `CONCURRENCY` 2→1 — this also dramatically *improved* single-question latency (35-37s vs. the previous 150-500s+ under contention), likely a net throughput win despite losing parallelism. Stall-detection window tightened 15min→8min accordingly.

7. Judge positivity bias — discovered and being addressed

Finding: across the first ~870 rows of the 8-factor dataset, Judge totals clustered tightly near the ceiling — mean 76.3/80 (95.4%), stdev 3.4, 81% of all scores in the 75-80 band. The rubric was not meaningfully discriminating between answers of different quality.

Attempt 1 — stricter rubric language: rewrote the scoring-band descriptions (recalibrated so "competent/adequate" = 5-6 rather than 7-9, required explicit justification before awarding 9-10). Result: **no measurable effect** — 8 fresh completions under the new prompt still averaged 76.4/80 with the same tight spread. Conclusion: abstract calibration language does not override the model's systematic positivity bias as an LLM-judge, even with explicit anti-leniency instructions.

Decision: archived the ~435 rows generated before this point (`data/judge_training_v2_lenient_rubric_ARCHIVE.db`) and restarted the v2 dataset from scratch, so the eventual dataset uses one consistent standard throughout rather than repeating the earlier mixed-regime mistake.

Attempt 2 (in progress) — alternative judge models: - **Prometheus 2** (`prometheus-eval/prometheus-7b-v2.0`, open-source model specifically fine-tuned for LLM-evaluation tasks): pulled via Ollama (HuggingFace GGUF, Q4_K_M). Tested with our exact 8-factor prompt format on a real Q&A pair. Result: **produced a genuinely lower/more differentiated score (54/80 = 67.5%)** — promising evidence of less positivity bias — but **completely ignored our structured per-criterion output format**, responding with free-form prose and a single overall score instead. It's rigidly trained on its own template; not usable as a drop-in replacement without either abandoning the 8-factor breakdown or building a from-scratch integration around its native grading format. - **gemma2:9b**: format-compatible — followed the 8-factor structure perfectly. Initial read looked promising (self-reported totals of 54/80 and 57/80, vs. qwen2.5's usual ~76/80).

8. Parser bug found and fixed — corrects the gemma2 finding

Investigating the gemma2 result surfaced a real bug: gemma2's own 8 per-criterion scores for Answer A were `9,10,8,9,8,10,9,9` — which sum to **72** — but it then wrote `TOTAL: 54/80` (off by 18) in its own output. The parser (`utils/rubric_parser.py`) was trusting that literal self-reported "TOTAL: X/80" line over recomputing from the individual scores it had just parsed. **The apparent "less biased" gemma2 result was an arithmetic mistake in the model's own summation, not genuine stricter grading** — its real per-criterion judgments show the identical positivity pattern as qwen2.5 (72/80 = 90%, 75/80 = 94% once correctly summed).

Fix: `_extract_total()` now always recomputes the total as the sum of the 8 parsed per-criterion scores; the model's self-reported total line is never trusted. Retroactively corrected all 54 rows collected so far in `judge_training_v2.db` (recomputed `judge_total_*` / `verifier_total_*` from the already-stored per-criterion columns — no LLM re-calls needed). Effect on the qwen2.5 baseline was small (76.3 → 76.1 mean, 3.4 → 3.1 stdev) — confirming qwen2.5's original bias measurement was not itself an artifact of this bug, mostly gemma2's was.

Revised conclusion: neither qwen2.5 nor gemma2 show real improvement in positivity bias once measured correctly — both cluster at 90-95% of max at the true per-criterion level. Prometheus 2's result becomes moot for comparison purposes (it never produced parseable per-criterion scores in our format at all, so under the corrected parser its total is now correctly 0/80 - i.e. genuinely unusable in this schema, not a valid data point either way).

Methodological note for writeup: this is a good illustration of why per-criterion structured scoring is more trustworthy than asking an LLM for a single aggregate number, or trusting an LLM's own arithmetic even when it also shows its work — the individual judgments were fine, the model's own addition wasn't. Always recompute derived totals from raw component scores rather than trusting a model's self-reported aggregate.

9. Broader model search + decisive discrimination test

To test whether the positivity bias was specific to qwen2.5, four more format-compatible models were tested with the identical prompt on the same real Q&A pair (question_idx=3, "difference between ML and DL"):

Model	Answer A total	Answer B total
qwen2.5 (baseline)	~76/80 (typical)	~76/80 (typical)
gemma2:9b	72/80	75/80
llama3.1:8b	72/80	73/80
mixtral	72/80	76/80
CompassJudger-1-7B	72/80	75/80

All five models, across four different architectures/families, converged on the same 72-76/80 range on this example. This raised a methodology concern: testing one (evidently strong) Q&A pair repeatedly only shows whether models agree that a good answer is good - it says nothing about whether they can tell a BAD answer apart from a good one.

Decisive test: constructed one genuinely broken answer (factually wrong, contradictory, irrelevant claims) alongside a solid one, same question, tested on qwen2.5 and CompassJudger-1:

Model	Good answer	Deliberately wrong answer	Gap
qwen2.5	73/80	**6/80**	67 pts
CompassJudger-1	64/80	**19/80**	45 pts

Conclusion: both models discriminate correctly when there's a genuine flaw to catch - the earlier "positivity bias" framing was partly a methodology artifact (repeatedly testing one strong example). The real question is which model is better *calibrated* on the space of realistic, mostly-competent answers this pipeline actually generates (LLaMA3/Mistral rarely produce answers as broken as the synthetic test case).

10. Head-to-head validation, N=40 real scores — decision made

Ran `training/compare_judge_models.py`: 20 real (question, answer_a, answer_b) triples sampled from the archived dataset, scored by both qwen2.5 and CompassJudger-1 under identical conditions (40 total scores per model).

Model	Mean	Stdev	Min	Max	% scoring ≥75
qwen2.5	71.9/80 (89.9%)	3.7	64	**80**	20.0%
CompassJudger-1	70.0/80 (87.5%)	3.8	60	**73**	**0.0%**

CompassJudger-1 never once scored ≥75 across 40 real evaluations, and its observed ceiling (73) sits meaningfully below qwen2.5's (which hit the absolute max of 80 one in five times). Combined with Section 9's finding that it still correctly penalizes genuinely wrong content, this is real, N=40 evidence of a measurable (if not dramatic) calibration improvement.

Decision: switched the Judge from qwen2.5 to CompassJudger-1 (`agents/judge_agent.py::_get_auditor_model`, 2026-08-18). Per the project's standing rule that any judge/model change gets a fresh dataset (established after the earlier mixed-judge-regime lesson - see Section 1), archived the qwen2.5-judged partial dataset (`data/judge_training_v2_qwen25judge_ARCHIVE.db`, 55 questions) and restarted generation from scratch. Verifier remains phi3 for now (not re-tested against alternatives yet).

11. Stricter rubric v3 (on top of the CompassJudger-1 switch)

Even after switching to CompassJudger-1, requested the scoring language be tightened further. `agents/rubric.py::SCORING_GUIDE` was rewritten ("v3"): narrower bands (adequate=4-5 instead of 5-6, strong=6-7 instead of 7-8), and a mandatory step requiring the model to actively search for a specific flaw before awarding any score ≥7, capping the criterion at 6 if one is found. Applies to both Verifier and Judge since they share `rubric.py`.

Per the project's standing rule (never mix scoring regimes in one database), the in-progress CompassJudger-1/v2-rubric run (12 rows) was retired and a fresh database started for CompassJudger-1 + rubric v3.

Process note for the record: during the restart, a second/orphaned `watchdog_v2.sh` instance (a recurring class of bug in this project - see Section on infrastructure issues) silently auto-restarted the generator before the intended clean restart, and an attempt to archive the 12-row interim database lost that data (copied the `.db` file without checkpointing WAL first, then deleted the WAL). Net effect: those 12 CompassJudger-1/v2 -rubric rows are unrecoverable, but since they were an early validation batch (not yet analyzed), this has no research impact. The rogue watchdog was killed and generation resumed cleanly under a single supervised instance. First 5 real rows under CompassJudger-1 + rubric v3:

Question	Judge A	Judge B	Verifier A	Verifier B	Weighted
CRISPR-Cas9	77	76	77	76	0.853
Quantum entanglement	75	74	75	74	0.717
ML vs DL	73	74	73	74	0.751
Neural network learning	76	74	76	74	0.692
Turing test	73	76	78	77	0.804

Scores are still in the 73-78 range on this small sample - consistent with the Section 9-10 finding that rubric-wording changes alone move the needle less than the underlying judge model does. Will need a larger sample before concluding whether v3 has any measurable effect on top of the CompassJudger switch; flagged as an open question rather than a settled result.

12. Anchoring bug: the Judge was not actually independent

After Section 11's rubric tightening, checked whether identical scoring language for Verifier and Judge could explain their close scores. Query against the live database (37 rows) showed something worse than shared language:

```
verifier_total_a, judge_total_a, verifier_total_b, judge_total_b, agreement
(77, 77, 76, 76, 1.0)
(75, 75, 74, 74, 1.0)
(73, 73, 74, 74, 1.0)
... [32 of 37 rows had verifier_judge_agreement EXACTLY 1.0]
```

32/37 rows (86%) had Judge totals matching Verifier totals **exactly**, digit-for-digit across all 8 factors for both answers - not just similar, identical. Two genuinely independent 8-factor evaluations landing on the exact same 16 numbers that often is not plausible.

Root cause found in `agents/judge_agent.py::build_judge_prompt`: the function received the Verifier's full raw output (including its per -criterion numeric scores) and inserted it directly into the Judge's prompt as `"VERIFIER'S INDEPENDENT SCORES (a 3rd-party opinion, not ground truth)"`. The system prompt told the Judge to "form your own independent judgment," but at temperature=0.1 with identical rubric anchoring, the Judge was overwhelmingly reproducing the shown numbers rather than re-deriving them. This meant `verifier_judge_agreement` (10% of the weighted-confidence formula) had been measuring anchoring/copying, not genuine second-opinion agreement, for the entire CompassJudger-1 run so far (and likely the qwen2.5-judge era too, since this bug predates the model switch).

Fix: `build_judge_prompt()` no longer takes or includes the Verifier's output at all - the Judge now scores completely blind from the question and the two answers, matching what the module's own docstring always claimed the design did. `verifier_judge_agreement` is still computed the same way afterward (comparing the two already-generated, now genuinely independent score sets), so the pipeline's downstream code and DB schema are unchanged - only the Judge's input context changed. Updated call sites in `generate_training_data_v2.py`, `server.py`, `generate_training_data.py`, `main.py`. Kept the Verifier→Judge execution order sequential (not made concurrent) despite the two now being independent, to avoid reintroducing the VRAM over-subscription stalls documented earlier in this log.

Per the no-mixed-regimes rule, archived the 40-row anchored-judge database (`data/judge_training_v2_anchoredjudge_ARCHIVE.db`) and restarted fresh. Open question for a future section: re-check `verifier_judge_agreement` once enough blind-scored rows exist - a healthy independent signal should show real spread (not clustering near 1.0), and a persistently high agreement post-fix would be a genuine (not artifactual) finding.

13. Rubric v4: per-criterion descriptive anchors + anti-default rule

After the Section 12 blind-judge fix, the Judge's totals still showed a suspicious pattern in the first 5 rows: 72 appeared as the Judge's total 3 times across 3 unrelated questions (CRISPR, quantum entanglement, ML vs DL). That's consistent with a "comfortable default" value the model falls back to regardless of content, rather than content-driven scoring - a different failure mode than Section 12's anchoring bug, but with the same symptom (suspiciously repeated numbers).

Hypothesis: `agents/rubric.py::FACTORS` gave each of the 8 criteria only a generic one-line description ("claims are accurate", "well-structured and easy to read") - the SAME vague standard applied to all 8, which gives a language model little concrete basis to differentiate a 4 from a 6 from an 8 on any specific criterion, inviting a memorized/default number.

Change ("v4"): rewrote every entry in `FACTORS` to state a concrete, criterion-specific differentiator - e.g. Completeness's mid-band is now "covers the obvious parts but skips a sub-part, an edge case, or a caveat a subject-matter expert would expect" instead of a generic quality judgment. Also added an explicit ANTI-DEFAULT rule to `SCORING_GUIDE`: before finalizing a TOTAL, the model must be able to point to a specific sentence justifying that exact number for that exact question, and repeated identical totals across unrelated answers are flagged as suspicious rather than assumed correct.

Per the no-mixed-regimes rule, archived the blind-judge/v3-rubric database (5 rows, `data/judge_training_v2_blindjudge_v3rubric_ARCHIVE.db`) and restarted fresh under rubric v4. This is a small, fast-moving set of changes (Sections 11-13 all landed within about an hour) - each individual change has too little data to draw firm conclusions yet; the goal for now is to get enough v4 rows to check whether the "72 pinning" pattern actually goes away, and to eventually go back and analyze whether v3 vs v4 made a measurable difference once volume allows.

14. CompassJudger-1 abandoned: it wasn't discriminating at all

Investigated the "72" pattern flagged in Section 13 by looking at CompassJudger -1's PER-CRITERION scores, not just totals. Found something worse than a default-value habit: on most answers it gives **all 8 criteria the exact same score** (e.g. 9/9/9/9/9/9/9/9), with boilerplate justification sentences that were sometimes IDENTICAL between Answer A and Answer B despite different content. Quantified the flat-rate (all 8 scores identical) across every CompassJudger-1 batch collected:

Dataset (CompassJudger-1 as Judge)	n	% all-8-identical
Anchored on Verifier's scores (Section 12 bug, pre-fix)	40	10%
Blind, rubric v3	5	80%
Blind, rubric v4 (descriptive factors)	4	75%
Blind, rubric v4, generation continued	43 (cumulative)	~74%

The rate got WORSE after fixing the anchoring bug, not better - when anchored, CompassJudger-1 was inadvertently inheriting the Verifier's real variation; scoring genuinely independently exposed that its own default behavior is to write plausible-sounding boilerplate around one flat number. Rubric v3→v4's more descriptive per-criterion language (Section 13) made no real difference (75% vs 80%, both on tiny n). **Conclusion: CompassJudger-1 -7B was not fit for this task** - the original N=40 validation (Section 10) that motivated adopting it only checked TOTALS and mean/stdev/ceiling-rate, which never would have surfaced this, since a model that outputs the same constant per-criterion score across most answers still produces some spread in the total simply from which flat value it picks per question. That validation methodology gap is itself worth remembering for future model comparisons: check per-criterion variance, not just the total.

Considered CompassJudger-1-32B (mradermacher GGUF, Q4_K_M ≈19GB) as a "bigger model, more capacity to actually reason" fix. Checked actual VRAM headroom first: `ollama ps` during a live pipeline run showed CompassJudger -7B + phi3 + mistral simultaneously resident at ~21GB/23GB (Ollama's `keep_alive` keeps recently-used models warm rather than unloading between pipeline stages) - a 19GB model would not coexist with the others without constant eviction/reload thrashing, the same VRAM over-subscription failure mode already fixed once in this project (see the CONCURRENCY=1 note in generate_training_data_v2.py). Started the download to test empirically anyway, but the user asked to stop it before completion given the risk; the partial download was deleted, no data lost (nothing had been generated with it).

Also tried Prometheus-2 (prometheus-eval/prometheus-7b-v2.0, already available locally, purpose-built for rubric-based LLM evaluation). It ignores the pipeline's 8-criterion structured format entirely and writes free-form prose ending in its own native `[RESULT] N` single-score format - architecturally incompatible with this pipeline's per-criterion parsing without a substantial redesign. Deprioritized without further testing.

15. qwen3.5:9b adopted - and a hybrid-reasoning pitfall

Tried `qwen3.5:9b` (Alibaba's newer hybrid-reasoning small model, pulled from Ollama's official library). First attempt through the production `BaseAgent`/LangChain wrapper returned a **completely empty response** - not a parse failure, an actually empty string. Diagnosed directly via `ollama.generate()`: `done_reason='length'`, `eval_count=900` (the full token budget), response=`''`. qwen3.5 is a hybrid-reasoning model with an internal chain-of-thought phase; for a long, complex prompt (the full rubric + two answer texts), it can consume its ENTIRE `num_predict` budget on internal thinking and never emit the actual answer. A trivial "2+2" prompt worked fine and still used 259 tokens of thinking - confirming this model always reasons internally by default, and the judge prompt is just long enough to exhaust the budget before any visible output.

Fix: Ollama's Python client accepts `think=False` to disable this. Confirmed it collapses token usage from 259→10 on the trivial prompt and produces real, complete output on the full judge prompt. Traced this through to LangChain's `OllamaLLM`, which exposes the same control as a `reasoning: bool | None` field (not obviously named - found by inspecting `OllamaLLM.model_fields` directly, not from any doc). Added a `reasoning` field to `agents/base_agent.py::BaseAgent` (passed straight through to `OllamaLLM`, `None` by default so it's a no-op for every other agent/model in the pipeline) and set it to `False` specifically for qwen3.5 in `agents/judge_agent.py::create_judge_agent`.

With thinking disabled, ran the same blind, production-prompt evaluation on 20 real Q&A pairs (via `ollama.generate` directly, then re-verified through the actual `create_judge_agent()` factory end-to-end):

- **Flat-rate (all 8 criteria identical): 0/20 = 0%** (vs CompassJudger-1's ~75-80%)
- Score range 50-77/80, mean 67.8/80 (84.8%) - real spread, not clustering
- Genuine per-answer differentiation, e.g. Panama Canal question: 50 vs 77 (a 27-point gap reflecting an actual quality difference between answers)
- Latency 8-24s per answer - comparable to CompassJudger-1, no throughput cost from the switch

Decision: switched the Judge to qwen3.5:9b (reasoning disabled), replacing CompassJudger-1. This is now the third judge model in this project's history (qwen2.5 → CompassJudger-1 → qwen3.5:9b), each swap driven by a different, progressively more granular failure mode: qwen2.5 had ceiling-hugging positivity bias (Section 6-10); CompassJudger-1 fixed the bias but turned out not to be discriminating between criteria at all (Section 14); qwen3.5:9b's only issue was a model-family-specific generation-config gap (thinking mode), not a scoring-quality problem.

Per the no-mixed-regimes rule, archived the 43-row CompassJudger-1 database (`data/judge_training_v2_compassjudger_flatscores_ARCHIVE.db`) and will restart fresh under qwen3.5:9b once generation is resumed.

Process note: while restarting, discovered FOUR separate orphaned `watchdog_v2.sh` instances running simultaneously (each independently auto-restarting the generator whenever any one of them stopped it), causing repeated duplicate-process spawns and a locked database file during archiving. This is the same class of bug noted earlier in this log (`TaskStop` not reliably killing the tracked shell), but had accumulated to 4 concurrent orphans rather than 1-2, most likely from the several pause/resume cycles during this investigation. All four were found and killed directly via PID (not `TaskStop`, which only tracks one of them) before the database could be safely archived. Worth remembering: after any pause/resume cycle, check `ps aux | grep watchdog_v2` for duplicates before trusting a single `TaskStop` cleared everything.

16. N=40 confirmation: qwen3.5:9b vs CompassJudger-1-7B

Ran the full head-to-head validation (20 real Q&A pairs, both models, blind, production rubric v4, `reasoning=False` for qwen3.5) to confirm Section 15's smaller sample at proper scale:

Metric	qwen3.5:9b	CompassJudger-1-7B
n	40	40
Mean	65.5/80 (81.9%)	69.3/80 (86.7%)
Stdev	9.5	9.3
Min / Max	19 / 78	16 / 73
% scoring ≥75	10.0%	0.0%
% flat (all 8 criteria identical)	**0.0%**	**57.5%**

Confirms Section 14's finding at full scale: CompassJudger-1 gives all 8 rubric criteria the exact same score on the majority (57.5%) of evaluations - the similar stdev between the two models is misleading on its own, since CompassJudger-1's spread comes from which single flat value it happens to pick per question, not from genuine per-criterion analysis. qwen3.5:9b never produced a flat score across all 40 evaluations.

Final decision: qwen3.5:9b (reasoning disabled) is the production Judge, replacing CompassJudger-1-7B. Ready to resume the 5000-question generation run fresh under this model once given the go-ahead (paused per user request while this investigation was ongoing).

17. Position/identity bias: LLaMA 3 was always "Answer 1"

User question prompted a check that had never actually been verified in this project: was the assignment of which model's answer goes into "Answer 1" vs "Answer 2" randomized to cancel out position bias? It was not. Confirmed in code:

- `generate_training_data_v2.py`: `answer_a, answer_b = ...(llama3_agent, mistral_agent)` - LLaMA 3 was unconditionally position A, Mistral unconditionally position B, for every single question in the project's history.
- Worse than plain position bias: the Verifier and Judge system prompts hardcoded the labels `"Answer 1 (LLaMA 3)"` / `"Answer 2 (Mistral)"`, and the per-call prompt text said `"--- ANSWER 1 (LLaMA 3) ---"` / `"--- ANSWER 2 (Mistral) ---"` - the model was explicitly told which brand wrote which answer, every time. Position and identity were both confounded, always in the same direction, for the whole dataset.

Checked win-rate (A vs B) across every dataset collected so far:

Dataset	Judge	A (LLaMA3, pos 1)	B (Mistral, pos 2)
lenient_rubric	qwen2.5	60.6% (264/436)	39.4%
qwen25judge (fixed rubric)	qwen2.5	66.1% (37/56)	33.9%
anchoredjudge	CompassJudger-1	42.5% (17/40)	57.5%
compassjudger_flatscores	CompassJudger-1	27.9% (12/43)	72.1%

The majority direction FLIPS between judge models (qwen2.5 favored position 1/LLaMA3 by 60-66%; CompassJudger-1 favored position 2/Mistral by 58-72%). If this reflected a genuine LLaMA-3-vs-Mistral quality gap, different judges should broadly agree on which model wins more often, with the margin varying - not flip which model is "better" entirely. This pattern is the signature of judge-specific position/identity bias confounding the win-rate signal, not a real quality difference. This means every "winner" statistic collected in this project to date is unreliable as a measure of LLaMA-3-vs-Mistral quality - it's at least partly measuring each judge's own position/brand preference instead.

Fix, four layers: 1. `agents/judge_agent.py`, `agents/verifier_agent.py`, `agents/combiner_agent.py`: stripped all "(LLaMA 3)" / "(Mistral)" labels from every prompt (both the static system-prompt output template and the per-call "ANSWER 1/2" headers). These agents now never see which model produced which answer - only "Answer 1" / "Answer 2", genuinely blind to identity. 2. `generate_training_data_v2.py::run_one_attempt`: after generating both answers, `random.random() < 0.5` decides whether LLaMA 3 or Mistral lands in position A for THIS question. Every question gets an independent coin flip. 3. New DB columns `model_a` / `model_b` (values `"llama3"` / `"mistral"`) record which physical model was actually in each position, so true per-model win rates remain fully recoverable by joining on this column - nothing is lost, the confound is just no longer baked into the agents' inputs. 4. `server.py` (the live interactive single-query pipeline) fixed as a follow-up: added a `remap_ab()` helper (`utils/rubric_parser.py`) that swaps every `<x>_a` / `<x>_b` key pair (and flips `winner`) in a parsed score dict. Per attempt, `server.py` now randomly decides which answer the Verifier/Judge see as "Answer 1" internally (independent of the fixed UI display order), then immediately remaps their parsed output back to UI order right after parsing - so every line downstream (`total_winner`, `winner_answer`, the `confidence` SSE event, the combiner call) is untouched and still correctly assumes answer1=LLaMA3/answer2=Mistral, while the Judge itself scored a randomized, brand-blind pair. The human-facing stream still shows LLaMA 3 as "Stage 1" and Mistral as "Stage 2" exactly as before - only the Judge's internal view changed.

Per the no-mixed-regimes rule, archived the 5-row position-biased qwen3.5 database (`data/judge_training_v2_qwen35_positionbiased_ARCHIVE.db`) and restarted fresh with position randomization active. Once enough position-randomized data accumulates, worth re-running the win-rate check (A/B should now land near 50/50 in aggregate if the bias hypothesis is correct, and any residual skew after grouping by `model_a` / `model_b` would be a more trustworthy signal of genuine LLaMA-3-vs-Mistral quality difference).

18. Checkpoint analysis: 670/5000 questions under the fixed pipeline

First substantive look at the dataset since all of Sections 12-17's fixes landed (blind Judge scoring, qwen3.5:9b, rubric v4, position/identity randomization). Snapshot at 670/5000 final questions (765 total attempts logged, all under one consistent regime - no mixed-regime rows).

Pipeline health - Regeneration rate: 14.2% of questions needed at least one retry (95 extra attempts / 670 questions) - healthy, not excessive. - Only 2.4% of final answers land below the 0.68 confidence threshold - the regeneration loop is doing its job; most low-confidence attempts get caught and retried rather than shipped. - 73s average per attempt (median 67.5s), consistent with the no-contention baseline established earlier in the session.

Score distributions

	Judge (qwen3.5:9b)	Verifier (phi3)
Mean	66.6/80 (83.3%)	74.0/80 (92.5%)
Stdev	6.8	2.7
% scoring >=75	7.5%	-
Min / Max	0* / 79	10 / 80

Judge mean (66.6) is close to the earlier N=40 validation's 65.5 (Section 16) - production behavior matches the pre-deployment test, a good sign the validation was representative rather than a fluke.

New finding: the Verifier (phi3), never tested, shows the same clustering pattern the whole Judge investigation was chasing. Mean 92.5% with stdev only 2.7 is the same "positivity clustering" signature as qwen2.5's original bias (Section 1-10). Only the Judge model has ever been swapped/validated for this; phi3 has run unchanged and unexamined since the start of the project. Flagged as an open candidate for the same kind of scrutiny (per-criterion flat-rate check, alternative model comparison) - not yet acted on, pending direction.

Flat-rate in production confirms the fix: 0.1% (1/1340 individual scores) - matches the N=40 pre-deployment validation's 0%, versus CompassJudger-1's 57.5% at the same check (Section 16). The single biggest quality change from this session's work.

Position/identity bias re-check (post-fix): win rate by raw position is still skewed (A 40.6% / B 59.4%), but by TRUE model identity it's much closer to balanced (LLaMA 3 46.9% / Mistral 53.1%). The GAP between these two splits (position skew larger than identity skew) suggests a residual, smaller bias independent of brand: the Judge leans toward whichever answer it sees SECOND (position 2), on top of a real but modest ~6-point Mistral quality edge. This is a cleaner result than anything possible before Section 17's fix, since position and identity are no longer confounded together - worth reporting as a genuine (if secondary) finding rather than dismissing as noise, given n=670.

Fact-check signal coverage: Wikipedia matched 94% of winning answers (avg relevance-gated score 0.685); Wikidata matched only 51% (avg 0.496). Matches the known limitation noted earlier (Section on wikidata_check.py) that Wikidata's label-matcher search is structurally weaker than Wikipedia's full-text search for extracting relevant entities from free-text answers - not a new problem, just now quantified at scale.

Parser edge case found (rare, informational): one row ("What is epigenetics?") has `judge_total_b=0`. Not a real score - qwen3.5 visibly went into a self-correcting arithmetic loop in its own output ("Still over... let's lower X to hit exactly 80") and switched to shorthand labels (`QR: 10`, `CR: 9`) instead of the required `- Question Relevance: X/10` format for Answer 2's section, which the regex parser doesn't recognize - defaults unmatched criteria to 0. The model's actual intended total was ~80/80, not 0. Rate: 1/670 = 0.15%, too rare to justify a parser or prompt change on its own, but recorded here so it doesn't look like an unexplained anomaly later.

19. Infrastructure: durable auto-restart via Windows Task Scheduler

Three separate multi-hour/multi-day generation gaps occurred (17 hours, 40 minutes, then 4 full days) where every pipeline process died - `keep_awake.py`, `viewer_server_v2.py`, `server.py`, the generator, and the in-session bash watchdog (`watchdog_v2.sh`) - leaving only Ollama itself running. Root cause: the watchdog and all core processes were launched from within a Claude Code session's background shell tree, which does not survive terminal closure, session teardown, or the machine sleeping - `keep_awake.py`'s `SetThreadExecutionState` calls only block *automatic* idle sleep, not every possible interruption (manual sleep, session loss, etc). Each gap was only caught the next time a monitoring check happened to run in an active session - not durable for an unattended multi-day run.

Fix: added `heartbeat.ps1` - checks whether each of the 4 core processes is running (`Get-CimInstance Win32_Process` + command-line match) and restarts any that are missing. Registered as a genuine Windows Scheduled Task (`AgenticLLM_Heartbeat`, via `schtasks /Create`, since `Register-ScheduledTask` hit an access-denied error in this environment) that fires every 5 minutes indefinitely, independent of any terminal or Claude Code session, plus once at logon. Verified end-to-end: manually triggered via `schtasks /Run`, confirmed result code 0 and a log entry in `heartbeat_log.txt`.

This is a coarser check than `watchdog_v2.sh` (which also detects a still-running-but-stalled process via DB/log staleness) - it only catches "process is completely gone," not "process is hung." That was sufficient for all three gaps actually observed (each was a total process death, not a hang), so it directly closes the failure mode that mattered without over-building. The in-session watchdog still runs as a second, smarter layer whenever a session is active.

20. Weight/threshold sweep: no ground truth, so label-free objectives

The weighted-confidence formula (`utils/scoring.py::DEFAULT_WEIGHTS` - judge 40%, similarity 15%, model_agreement 10%, verifier_judge_agreement 10%, wikipedia 15%, wikidata 10% - and `DEFAULT_THRESHOLD = 0.68`) had been a starting guess since the beginning of the v2 pipeline, never actually validated. Built `training/threshold_weight_sweep.py` to test it against the ~1765 attempt rows collected so far under the current (qwen3.5:9b, position-randomized) regime.

There is no ground-truth "this answer is objectively correct" label anywhere in this dataset, so two label-free objectives were used instead of accuracy-against-a-target:

1. Internal consistency: for each signal, build a composite from the OTHER signals only (properly renormalized) and correlate it against the held-out signal. A weighting under which signals predict each other well is more likely capturing real shared "quality" signal than one dominated by a single noisy weight.

2. Regeneration validity: for questions that got a 2nd attempt, check whether attempt 2 actually scored better on signals OTHER than judge (to avoid circularity) - a meaningful threshold should mostly trigger regeneration when attempt 1 truly was weaker.

Methodological catch found along the way: `verifier_judge_agreement` correlates 0.737 with `judge` in this data - expected, since it's mathematically computed FROM the judge and verifier totals (`compute_agreement()` in `utils/rubric_parser.py`). Weighting it heavily inflates the internal-consistency metric without adding real independent signal - it's partially a restatement of `judge`, not corroboration. The first sweep run (that included it as a free variable) produced misleading top candidates (e.g. verifier_judge_agreement at 0.78 weight) for exactly this reason. Re-ran with it excluded from the free search for a trustworthy result.

Clean weight-sweep result (5 genuinely independent signals: judge, similarity, model_agreement, wikipedia, wikidata):

	judge	similarity	model_agreement	wikipedia	wikidata	score
Default (renormalized)	0.44	0.17	0.11	0.17	0.11	0.1216
Best of 3000 random candidates	~0.25	~0.29	~0.09	~0.33	~0.03	0.1331

Directional finding: similarity and Wikipedia consistently want MORE weight than the default gives them; wikidata wants LESS (converging near 0) - consistent with Section 18's finding that Wikidata only matches 51% of questions with a noisier average score. Judge's weight comes down somewhat but stays substantial. Effect size is real but modest (+0.011, ~9% relative improvement in the consistency metric) - a hypothesis worth testing further as more data accumulates, not a confident final answer. Weights have NOT been changed in production pending more data.

Threshold sweep hit a harder limitation: selection bias / censored data. The sweep is only informative BELOW 0.68 - above it, results are identical for every threshold tested (184 flagged, 0 not-flagged), because the live pipeline only ever generates a 2nd attempt when attempt 1 scored below 0.68. There is no data on whether an ABOVE-threshold first attempt would also have improved on retry, because it's never been given the chance. Sub-0.68 data shows retry-improvement rates hovering 77-89% across the 0.58-0.66 range with no sharp cliff - 0.68 doesn't look obviously wrong, but this data can't confirm it's optimal either.

21. Control-group regeneration added to close the threshold blind spot

To fix Section 20's censored-data problem properly (not just note it), added a genuine control group to `generate_training_data_v2.py`: when a first attempt already scores >= 0.68 (i.e. would normally ship immediately), there is now an 8% chance (`CONTROL_REGEN_PROBABILITY`) it gets a 2nd attempt anyway, purely to measure whether already-passing answers also tend to improve on retry. This is additive and safe for production quality - the existing "keep whichever attempt scored higher" logic (`process_question`'s `best` tracking) is untouched, so a control retry can never make a shipped answer worse, only occasionally produce an even better one that gets kept.

Each attempt row now records WHY a regeneration happened, in a new `regen_reason` column (`'low_confidence'` for a genuine sub-threshold retry, `'control_sample'` for this new control group, `NULL` for a normal single-pass question) - added via `ALTER TABLE ... ADD COLUMN` on the live database (backward compatible; the ~1765 existing rows just have NULL). No archive/restart needed since this doesn't change how any individual attempt is scored, judged, or ranked - only whether a rare extra attempt gets logged for research purposes on top of the existing consistent regime.

At 8% of roughly 3500 remaining questions, this should accumulate a few hundred control samples by the time generation finishes - enough to finally answer, with real (not selection-biased) data, whether the 0.68 threshold is well-calibrated, too conservative (control samples rarely improve, meaning many low-confidence retries below it are also probably wasted), or too lax (control samples improve about as often as genuine sub-threshold retries do, meaning the threshold should be raised).

22. First control-group read: 0.75 looks better than 0.68 (n still small)

With 53 control samples accumulated (Section 21's mechanism), ran the first genuinely unbiased comparison of retry-improvement rates:

Group	n	Retry improves other signals (judge excluded)
Genuine sub-threshold (score < 0.68)	144	66.7%
Control sample, score 0.68-0.75	31	64.5%
Control sample, score 0.75+	22	40.9%

The 0.68-0.75 band behaves almost identically to genuine sub-threshold cases (64.5% vs 66.7%) - these are "passing" answers nearly as improvable as ones already flagged for regeneration. The 0.75+ band drops sharply to 40.9%, looking like a genuinely settled "good enough" zone. This is clean, monotonic, first-time-unbiased evidence that **0.75 would be a better DEFAULT_THRESHOLD than the current 0.68** - right now the pipeline ships a meaningful fraction of still-improvable answers without a second look purely because they cleared an apparently-too-low bar.

Not yet acted on. n=31/n=22 per band is small (~±17pp margin of error at 95% CI) - directionally convincing but not yet a safe basis to change the live threshold. Decision: let the control-group mechanism keep running (~200 more expected by the time the 5000-question run finishes) and re-check before committing to 0.75. If the gap holds up at n~150-250 per band, this becomes a strong, well-supported case for the change.

23. Threshold re-check with statistical rigor: significant, but costly to act on

Requested re-analysis at 3536/5000 questions (117 control samples, more than double Section 22's 53). Built `training/threshold_confidence_analysis.py` to add what the earlier read lacked: Wilson score confidence intervals per band and a two-proportion z-test between the two control bands, instead of comparing raw percentages by eye.

Band	n	Retry improves other signals	95% CI
< 0.55	41	68.3%	[53.0, 80.4]
0.55 - 0.62	109	61.5%	[52.1, 70.1]
0.62 - 0.68 (sub-threshold)	492	65.7%	[61.3, 69.7]
0.68 - 0.75 (control)	77	61.0%	[49.9, 71.2]
0.75+ (control)	41	36.6%	[23.6, 51.9]

z = 2.533, two-tailed p = 0.0113 for the 0.68-0.75 vs 0.75+ gap - this clears the ~150-250-sample bar loosely and, worked out properly, needed only ~32 samples per band to reach significance at this effect size; the 77/41 in hand comfortably clears that. **This is now a real, statistically significant finding, not a directional one**: answers already scoring 0.68-0.75 improve on retry about as often as genuine sub-threshold answers (61.0% vs 62-68% across the three bins below 0.68 - no sharp cliff there either), while answers scoring 0.75+ improve much less often (36.6%). The information the confidence score carries about "is this worth a second look" doesn't actually run out at 0.68 - it keeps discriminating up to around 0.75.

Second catch, found while re-running the weight sweep on the larger dataset: `training/threshold_weight_sweep.py`'s top candidates were once again dominated by heavy `verifier_judge_agreement` weight (up to 0.78) - the exact circularity Section 20 already diagnosed and reportedly excluded "from the free search," but the fix had only ever been applied ad hoc in an interactive session, never actually committed to the script. Fixed properly this time: added `INDEPENDENT_KEYS` (the 5 non-derived signals) and threaded a `keys` parameter through `weighted_composite` / `internal_consistency_score` so verifier_judge_agreement is excluded both as a free weight AND as a held-out target (excluding it only as a free weight would still leak the correlation back in through the leave-one-out average). The corrected sweep is the one worth trusting going forward; ad hoc workarounds that never make it back into the checked-in tool are exactly the kind of thing this log-keeping habit exists to catch.

Why this isn't being acted on immediately anyway - the real cost is operational, not statistical. Computed the actual regeneration-rate impact across all 3544 first attempts collected so far:

Threshold	First attempts that would pass	Regeneration rate
0.68 (current)	81.8%	18.2%
0.70	71.6%	28.4%
0.71	65.4%	34.6%
0.72	58.0%	42.0%
0.74	43.1%	56.9%
0.75	35.5%	**64.5%**

Moving to 0.75 would raise the regeneration rate from 18.2% to 64.5% - a ~3.5x jump - for every question generated from this point forward. There is no cheap middle ground either: even a small step to 0.70 nearly doubles it (18.2% -> 28.4%). At the project's established ~73s/attempt baseline (Section 18), this would substantially slow completion of the remaining ~1460 questions. Unlike every prior judge/rubric change in this log, this particular change does NOT invalidate or alter any already-stored score - every signal is computed identically regardless of `DEFAULT_THRESHOLD`, so raising it mid-run would not be a mixed-scoring-regime problem in the sense Sections 1, 10, 12, 13, 15 and 17 were - only a mixed-*bar-for-shipping* one, and a fully recoverable one at that (the `regen_reason` column already records enough to reconstruct, post hoc, which threshold each row would have shipped under).

Decision: hold at 0.68 for the remainder of this run, given the completion-time cost, and treat 0.75 as the evidence-backed recommended default for the *next* dataset generation run or production deployment where completion-time pressure doesn't apply the same way. Flagged back to the user as an open decision rather than changed unilaterally, since the tradeoff (dataset quality bar vs. time-to-5000) depends on project timeline constraints outside this analysis's visibility.

24. Optimal threshold, proven properly: ROC/Youden's J + bootstrap → 0.75 adopted

User asked for actual proof that a candidate threshold is the best one possible, not just "better than 0.68." Built `training/optimal_threshold_roc.py` to frame this as a standard binary-classifier-cutoff problem: attempt 1's `weighted_score` is a predictor, the target it's predicting is "would this answer actually improve if regenerated" (same OTHER-signals-only, judge-excluded definition used since Section 20), and every question that ever got a 2nd attempt provides one labeled (score, improved) pair.

A real methodological mistake, caught before it went in the record: the first version computed sensitivity/specificity/AUC directly from raw labeled rows and got a nonsense result - AUC=0.5079 (indistinguishable from random) and an "optimal" cutoff of 0.66, *below* the current threshold, flatly contradicting Section 23's significant finding. Cause: this dataset is not a simple random sample. Every sub-threshold attempt is regenerated (100% census), but only 8% of at-or-above-threshold attempts are (`CONTROL_REGEN_PROBABILITY`, Section 21) - a case-control / stratified -sampling design. Raw counts silently treat a control-sample row as equally representative as a census row, undercounting the true above-threshold population by ~12.5x and corrupting every population-composition-dependent statistic (sensitivity, specificity, AUC) even though it does NOT bias the simple per-band rates Section 23 used. Fixed with inverse-probability-of -sampling weighting - the standard correction for exactly this design in diagnostic-test validation statistics: every low_confidence row weight 1.0, every control_sample row weight 1/0.08 = 12.5x.

Corrected result:

Method	Result
Two-proportion z-test, 0.68-0.75 vs 0.75+ bands (Section 23)	p = 0.0113
Sampling-corrected ROC AUC	0.6211 (real signal, not random)
Sampling-corrected Youden's J optimum	**t = 0.74** (sensitivity 79.1%, specificity 39.8%, J=0.1885)
2000-resample bootstrap of the optimum	median 0.73, **mode 0.75**, 95% CI **[0.69, 0.79]**

Three independent methods (a significance test, a corrected ROC/Youden's-J optimization, and a bootstrap stability check) converge on the same 0.73-0.75 neighborhood. The current 0.68 sits right at the edge of the bootstrap CI, not inside its bulk - i.e. it's more likely to be *miscalibrated low* than for this result to be noise.

Decision: `DEFAULT_THRESHOLD` **raised from 0.68 to 0.75** in `utils/scoring.py` (0.75 chosen as the bootstrap mode - a defensible, round value sitting inside the confidence interval, not just the raw Youden optimum of 0.74). `generate_training_data_v2.py` and `server.py` both restarted to pick up the new value (both import it as a module-level constant, so a running process would otherwise keep the old one in memory indefinitely). Per Section 23's own cost analysis, this raises the regeneration rate on the remaining ~1450 questions from ~18% to ~64.5% - accepted deliberately in exchange for a properly-proven, higher quality bar, rather than held for consistency as Section 23 initially recommended. `CONTROL_REGEN_PROBABILITY`'s 8% mechanism keeps running unchanged and will now sample the space above 0.75 instead of above 0.68, so this same analysis can be re-run later to check whether an even higher cutoff is justified once enough new control data exists above the new bar.

25. RAG's MIN_SIMILARITY has never been calibrated - and currently can't be

User asked whether `utils/rag_store.py::MIN_SIMILARITY` (0.35, the cosine -similarity gate below which a retrieved chunk is dropped rather than injected as context) had gone through the same scrutiny as `DEFAULT_THRESHOLD`. It hadn't, and unlike the confidence threshold, it currently **can't** be - not a matter of insufficient sample size, but a complete absence of a data path.

Root cause: `generate_training_data_v2.py::run_one_attempt(query, rag_context="")` defaults to no RAG context, and the sole call site (`process_question`, line ~291) calls it as `run_one_attempt(question)` - never overriding `rag_context`. Every one of the 3500+ questions in the entire calibration dataset ran with RAG **completely inert**. `MIN_SIMILARITY` only ever executes inside the live interactive demo (`server.py`), which sees a tiny fraction of the traffic the batch generator does, and logs no retrieval outcome anywhere persistent (no `regen_reason`-style column recording whether a kept/dropped chunk was actually useful). There is nothing in `judge_training_v2.db` to sweep this threshold against, at any sample size.

What was done instead - a plausibility spot-check, explicitly not a calibration: built `training/rag_threshold_spotcheck.py`, which queries the REAL, currently-populated knowledge base (7 documents / 98 chunks, `data/rag_knowledge.db` - the 2 defaults plus 5 PDFs uploaded through the live demo: a quantum physics paper, a bioinformatics perspective piece, and 3 solar-system documents) with 5 deliberately on-topic and 5 deliberately off-topic hand-written queries, reading the raw top-1 similarity before any gate is applied.

	Result
On-topic queries correctly passing the gate	5/5 (scores 0.501-0.623)
Off-topic queries correctly blocked	5/5 (scores 0.082-0.298)
Gap between weakest on-topic and strongest off-topic score	+0.202

0.35 sits cleanly inside the [0.298, 0.501] safe zone on this sample - it is not obviously broken. But this answers a much weaker question than the confidence-threshold work did: "is 0.35 obviously wrong?" (no), not "is 0.35 optimal, and by how much would a different value change outcomes?" (unknown - 10 hand-picked queries against 7 documents is a sanity check, not a statistically powered result, and has none of the real-usage grounding the control-group mechanism gave DEFAULT_THRESHOLD).

What a real calibration would need: the live demo would have to start logging every RAG decision (query, retrieved chunk text, raw similarity, whether it was kept or gated out) to a persistent table, plus some usable proxy for "was this retrieval actually good" - e.g. re-running `compute_weighted_score` on the same query with RAG on vs. off and comparing, mirroring the A/B structure the confidence-threshold control group used. That requires real interactive-demo traffic to accumulate, which this project has approximately none of compared to the batch generator's 3500+ logged questions - not started, pending direction.

26. RAG retrieval logging added, and a live "was RAG used" UI indicator

Two follow-ups to Section 25's finding that MIN_SIMILARITY has no data path to calibrate against.

Logging (`utils/rag_store.py`): `retrieve_context()` now logs every candidate chunk's raw similarity - not just the ones that pass the gate - to a new `rag_retrieval_log` table in `data/rag_knowledge.db`, tagged with a per-request `request_id`. A second table, `rag_pipeline_outcomes`, records that same `request_id` against the run's final `weighted_score` and whether `use_rag` was on, written once `server.py` finishes a query. Same raw-signals-first philosophy as `generate_training_data_v2.py` (Section 5): log everything now, decide what to sweep later. Verified end-to-end - one real query logged 98 rows (one per chunk in the knowledge base) tagged with a shared request_id.

UI indicator: the live demo previously received a `rag` SSE event but silently discarded it (`if (p.retrieved !== undefined) { return; }` - "informational, no dedicated card needed"). Wired it up: `server.py` now also emits the event when RAG was NOT used (previously only emitted on a successful retrieval), distinguishing two reasons - `disabled` (checkbox off) vs `no_relevant_document` (nothing in the knowledge base cleared MIN_SIMILARITY for this question). The frontend shows a badge next to the RAG toggle: "✓ RAG used · N chunks" or "○ RAG: no relevant document". Confirmed over the real SSE stream for all three cases (used / no-match / disabled).

This directly answers a user question about whether an uploaded document gets forced into every answer regardless of relevance: it doesn't - the MIN_SIMILARITY gate already excludes RAG for off-topic questions (verified again here, consistent with Section 25's spot-check), and now that's visible in the UI instead of only inferable from the network tab.

Also clarified: `MIN_SIMILARITY` (retrieval relevance) and `DEFAULT_THRESHOLD` (post-generation confidence, Section 24) are independent thresholds on unrelated scales - raising one has no formulaic bearing on the other. The one real open connection: since RAG never fires in the batch generator (Section 25), the 0.75 threshold was proven entirely on non-RAG answers, so whether it's equally well-calibrated for RAG-assisted ones is unknown. `rag_pipeline_outcomes` now collects exactly what a future check of that would need.

27. A side-effect of 0.75 not caught in Section 24: most final answers now ship below it anyway

User spotted rows in the dataset viewer where a question's kept ("final") attempt still scored below 0.75 and asked whether regeneration was actually working. Traced the exact logic (`generate_training_data_v2.py::process_question`): the retry loop is correctly comparing every attempt against the live `DEFAULT_THRESHOLD` (confirmed - e.g. a 0.751 attempt gets `regen_reason=None`/kept, a 0.742 one gets flagged `low_confidence`, exactly at the 0.75 boundary). Not a bug. But `MAX_REGENERATION_ATTEMPTS = 2` caps every question at 3 total attempts, after which the pipeline ships whichever of the 3 scored highest - even if NONE of them cleared the threshold. So "threshold 0.75" does not mean "every shipped answer scores >=0.75"; it means "try up to 3 times to reach 0.75, then ship the best attempt regardless."

Quantified for the 521 final answers shipped since the 0.75 change:

	Count	%
Shipped below 0.75 despite the retry loop	310	59.5%
...of those, exhausted all 3 attempts and still never cleared it	110	21.1% of all shipped

This is a real consequence of Section 24's threshold change that wasn't surfaced at the time - the regeneration-cost analysis (Section 23/24) measured how often FIRST attempts would need a retry, not how often the retry loop would actually succeed in reaching the new, harder bar within the existing 3-attempt cap. Worth remembering for the writeup: raising a threshold changes two things at once - how often retries trigger, AND (given a fixed attempt cap) how often they actually succeed - and only the first was measured before shipping the change.

Decision: accepted as-is. `MAX_REGENERATION_ATTEMPTS` stays at 2 (3 total attempts). Consistent with Section 23's ceiling finding (few answers ever exceed ~0.85, and per-attempt improvement rates decline the higher the bar climbs) - a 4th or 5th attempt would very likely keep paying compute cost for shrinking odds of actually clearing 0.75, not obviously a better trade than "best of 3." No code change. The dataset's `weighted_score` column remains the honest, ungated signal either way - `regen_reason` and `is_final_attempt` describe what the pipeline DID, not a guarantee about the number itself, and any downstream analysis should keep filtering on `weighted_score` directly rather than assuming "final" implies ">= threshold."

28. Dataset viewer fixed to show discarded attempts, plus a stale 0.68 reference caught

Direct follow-up to Section 27: a user spot-check of question #4070 in `data_viewer_v2.html` showed "Attempt 2 · 63.7%" with no sign a 3rd attempt ever ran, reading exactly like the regeneration loop had silently stopped early. It hadn't - `question_idx=4070` genuinely has 3 logged attempts (62.8% / 63.7% kept / 63.1% discarded) - the viewer's `/api/recent` only ever selected `is_final_attempt=1` rows, so a tried-and-discarded attempt was simply invisible, not missing.

Fix: `viewer_server_v2.py::get_recent()` now includes `total_attempts` per row (a correlated subquery) so the table badge reads "2/3" instead of a bare "#2". `get_detail()` now also returns `siblings` - every other attempt for the same `question_idx` - and the detail modal renders a new "Attempt history for this question" table listing every attempt tried, which one was kept, and why the others weren't. Verified against question 4070 directly: `/api/detail` now correctly returns the kept attempt (63.7%) alongside both siblings (62.8%, 63.1%).

Also caught while in this file: the row-coloring threshold (`weighted_score >= 0.68 ? 'score-ok' : 'score-low'`) was never updated when `DEFAULT_THRESHOLD` moved to 0.75 in Section 24 - every row scoring 0.68-0.75 was showing green ("passes") in the viewer despite actually being below the live threshold. Fixed to match. A reminder that a threshold living in two places (the scoring code and a UI's display logic) needs both updated together - worth checking for other such copies before the next threshold change.

29. Is the pipeline Mistral-biased? A real, pre-existing, modest edge - separable from a bigger position artifact

Direct user question, answered with data rather than assumption (n=4,091 final answers, current position-randomized regime):

	LLaMA 3	Mistral
By raw position (A vs B)	35.2%	64.8%
By true model identity	44.7%	55.3%

Both gaps are real (z~6.8 and z~18.9 respectively against a 50/50 null - not noise at this n). A proper 2x2 cross-tab (win rate for each model WITHIN the same position) separates them cleanly:

	Position A	Position B
LLaMA3 there	29.3% (n=1993)	59.3% (n=2100)
Mistral there	40.7% (n=2100)	70.7% (n=1993)

Mistral outperforms LLaMA3 by ~11 points in EITHER slot - not explained by lucky positioning. Ruled out length (Mistral answers are actually shorter, 313 vs 361 words) and topical relevance (LLaMA3 is slightly MORE similar to the question, 0.760 vs 0.733) as explanations.

Per-criterion localization: the Judge's gap concentrates almost entirely in three related criteria - Factual Correctness (+0.56/10), Faithfulness (+0.49), Hallucination-Free (+0.35) - while Clarity, Completeness, and Relevance show near-zero gap. Looked like a real, specific "Mistral is judged more factually sound" finding.

Then contradicted by the Verifier: phi3 scores the SAME answer pairs independently, and shows NO gap on any criterion (all ±0.03 or less). Two live explanations, not yet resolved: (a) the Judge's factual - correctness read is a style-driven illusion specific to qwen3.5:9b, or (b) the Verifier's known near-ceiling clustering (Section 18, still untested) makes it unable to detect a real gap either way. Recommended next step (not yet run): give the Verifier the same deliberately-broken -vs-solid discrimination test from Section 9, to check if it can detect ANY real quality gap at all.

Is this new (started "yesterday")? No - checked directly:

	LLaMA3	Mistral
Before the 0.75 threshold change	45.6%	54.4% (n=3556)
After the 0.75 threshold change	38.9%	61.1% (n=548)

The edge pre-dates the threshold change and was already significant at n=3556. It looks bigger now mainly because regenerated (best-of-2/3) answers have ALWAYS shown a slightly bigger Mistral edge than single-shot ones (58.1% vs 54.4%, consistent pre- and post-change), and the threshold change shifted the dataset's mix from ~18% regenerated to ~82% regenerated - an existing pattern getting more weight, not a new one appearing.

30. Viewer bug found via the Mistral-bias investigation: winner names were hardcoded to position, not identity

A user screenshot of `data_viewer_v2.html` showing ~23/24 recent rows won by "Mistral" prompted a direct DB check of the exact rows shown. The raw data told a different story: e.g. `question_idx=4104` has `model_a=mistral, model_b=llama3, winner=B` - the TRUE winner is llama3 - but the viewer displayed "Mistral."

Root cause: `data_viewer_v2.html` hardcoded `winner === 'A' ? 'LLaMA 3' : 'Mistral'` in three places (the recent-attempts table, the detail modal's meta line, AND the detail modal's answer-column titles - meaning the actual answer TEXT was mislabeled with the wrong model's name too). This assumption was true before Section 17's position-randomization fix (LLaMA3 was unconditionally position A back then) and was never updated once position started being assigned per-question by coin flip - a "the fix was applied to the scoring path but not the display path" gap that's existed since Section 17 shipped.

Practical impact: this specific screenshot's near-unanimous run happened to coincide with a stretch where the coin flip landed `model_a=mistral` repeatedly - the true win split in that window was much closer to the pipeline's normal 55-60% Mistral rate (confirmed against the raw DB), not the ~96% the display suggested. The Section 29 analysis above was computed directly from SQL joining `winner` against `model_a` / `model_b` correctly throughout, so it is NOT affected by this bug and stands as reported - only the VIEWER's on-screen labels were wrong.

Fix: `viewer_server_v2.py` now computes true-identity win counts (`wins_llama3` / `wins_mistral`) alongside the old position-only ones, and `get_recent()` now returns `model_a` / `model_b` per row. `data_viewer_v2.html` now derives every displayed model name (table winner, modal winner, BOTH answer-column titles, and the aggregate "winner split" stat) from `model_a` / `model_b`, never from raw position. Verified against the exact row that exposed the bug (`question_idx=4104`) - now correctly shows LLaMA3 as winner.

Process lesson for the record: this is the second time a fix landed in the scoring/generation path but not a downstream display path (the first was Section 24/27's threshold value living in two places). Worth a standing habit: after any change to `model_a` / `model_b`, `winner`, or threshold semantics, grep the viewer files too, not just the pipeline code.

31. Verifier discrimination test: it can tell, decisively - the Judge's Mistral edge stands unexplained-away

Direct follow-up to Section 29's open question: does the Verifier's ~0 LLaMA3-vs-Mistral gap mean "no real difference," or is phi3 just unable to detect any quality gap at all (its known near-ceiling clustering, Section 18)? Ran the same decisive test Section 9 used on the Judge back when qwen2.5/CompassJudger-1 were under investigation - `training/verifier_discrimination_test.py`, one genuinely solid vs. one deliberately broken (factually wrong, self-contradictory, irrelevant) answer per question, same production rubric/prompt:

Question	Solid	Broken	Gap
ML vs. deep learning	77/80	8/80	+69
CRISPR-Cas9 gene editing	80/80	0/80	+80

Average gap +74.5/80 - about as decisive as this test can get. **The Verifier absolutely can discriminate when there's a real quality gap to find.** This settles Section 29's open question: its near-ceiling clustering on ordinary (non-broken) answers is a real, separate pattern worth its own investigation some day, but it does NOT mean phi3 is incapable of registering differences - so its flat ~0 gap between LLaMA3 and Mistral is a genuine "these two models' answers looked equally good to an independent evaluator" reading, not an instrument-sensitivity artifact.

Net effect on the Mistral-bias question (Section 29): the Judge's concentrated ~0.5/10 edge on Factual Correctness / Faithfulness / Hallucination-Free now looks LESS likely to be real content quality and MORE likely a qwen3.5:9b-specific perception (style, phrasing, or some other confound) - a second, independent, and now demonstrably sensitive evaluator looked at the identical answer pairs and saw nothing. Not fully proven either way without a ground-truth fact-check per answer (which doesn't exist at this scale), but the balance of evidence shifted meaningfully toward "judge artifact" rather than "genuine Mistral advantage." Flagged as the leading open question for the next research phase, rather than settled.

32. Control-sample rate raised 8% -> 60% to actually finish the threshold recheck this run

At 4350/5000, checked how much control-sample data (Section 21's mechanism) had actually accumulated under the NEW 0.75 threshold specifically - not just in total. Of 133 total control_sample rows, only **16** were drawn from the post-0.75 population; the other 117 predate the threshold change and were already spent proving 0.75 itself (Section 24) - they can't be reused to test whether an even higher bar would be justified, since they were sampled from the old >=0.68 population, not >=0.75.

At the original 8% rate, only ~18 more were projected by the time the 5000-question run finishes (based on ~35% of first attempts passing at 0.75, times 8%, times the ~650 remaining questions) - nowhere near the 150-250 needed for a confident recheck, meaning this run would end with the threshold question still unresolved.

Fix: raised `CONTROL_REGEN_PROBABILITY` from 0.08 to 0.60. Only the ~35% of questions that already pass on attempt 1 are affected (the other ~65% already get a real regeneration attempt regardless), so this adds a modest ~8-10% more total attempts over the remaining ~650 questions, not a repeat of the much larger cost the original threshold change carried. Expected payoff: ~136 additional post-0.75 control samples, landing total around **~150** by completion - enough to actually re-run the Section 24 methodology (ROC/Youden's J + bootstrap) against a threshold-appropriate sample before this dataset run ends, rather than deferring it to a future one.

Considered and rejected: storing the new control-sample attempts in a separate database. The existing `regen_reason='control_sample'` column already isolates this data with zero extra code; splitting storage would touch the live write path of an 87%-complete run for no analytical benefit, and would require cross-database joins to compare a control attempt against its own question's attempt 1 (which would remain in the main database) - strictly worse than the single-database tag already in place. Generator restarted with the new probability; verified via `py_compile` before restart and a fresh PID after.

33. Dataset generation complete (5000/5000) - a real duplicate-final-row bug found and fixed at the finish line

Generation reached 5000/5000 questions. Completion checklist run before calling the dataset done:

Pipeline health: heartbeat log clean straight through completion, no gaps in the final stretch. Generator process exited on its own after question 5000 (confirmed via `training_log_v2.txt`'s last line and the process no longer running).

Data integrity check caught a real bug: `COUNT(*) WHERE is_final_attempt=1` returned 5003, not 5000 - 4 questions each had TWO rows flagged final. Traced to the exact mechanism: the final-marking statement matched rows by `(question_idx, attempt_number)` -

```
UPDATE pipeline_runs SET is_final_attempt = 1, ...
WHERE question_idx = ? AND attempt_number = ?
```

- which looks unique per question but isn't, whenever the generator gets killed between inserting an attempt and reaching this update (a crash, a sleep gap, or a deliberate restart for a code change - all of which happened repeatedly across this project's history, per Sections 6 and 19). The orphaned old row is left with no final flag; on restart, `already_done()` correctly doesn't find one and reprocesses the question from scratch, but the fresh run's own "attempt 1" shares that same `(question_idx, attempt_number)` pair with the orphan - so the UPDATE matches and flags BOTH rows. One of the four (`question_idx=3552`) lines up exactly with the restart performed for Section 24's threshold change - this session caused at least one of these four itself, not just inherited it.

Fixed both the data and the root cause, not just the symptom: - Un-flagged the 4 orphaned rows directly (kept whichever row belonged to the run that actually reached the Combiner step) - back to a clean 5000/5000. - `insert_attempt()` now returns its row's `id`; `process_question` tracks `best_row_id` alongside the best-scoring attempt and the final UPDATE targets `WHERE id = ?` instead of the ambiguous idx+attempt_number pair. This can't recur regardless of how many times a future run gets interrupted mid-question.

Final dataset stats (post-fix): 5000/5000 questions, 5000 final rows (verified duplicate-free), 8432 total attempt rows logged. Core signals (judge, similarity, model agreement, verifier-judge agreement) 100% available on every final row - zero nulls. Wikipedia/Wikidata coverage 53.9%/40.6% (optional, relevance-gated by design). Position randomizer held at 48.7%/51.3% llama3/mistral across the entire run - no drift. regen_reason tally: 4104 clean, 4163 low_confidence, **165 control_sample total, 48 of them post-0.75** (the population relevant to the final threshold recheck below).

34. Final threshold recheck at 5000/5000 - another sampling-weight bug caught before being reported, then a real answer

Ran `training/optimal_threshold_roc.py` against the complete dataset. First result looked alarming: optimal cutoff **0.68**, 95% bootstrap CI [0.68, 0.71] - appearing to flatly contradict Section 24's 0.75 decision.

Not trusted at face value - and rightly so. `CONTROL_REGEN_PROBABILITY` changed mid-dataset (0.08 -> 0.60, Section 32), but the script imported and applied *today's* value (0.60) to every `control_sample` row regardless of when it was generated - silently undercounting the ~117 rows actually sampled at the old 8% rate (weighting them 1.67x instead of their true 12.5x). This dilutes exactly the hard-won data that originally justified 0.75, pulling the result back toward the low_confidence -dominated answer the very first (pre-Section-24-fix) version of this script produced - the same failure shape as before, just from a different cause.

Boundary found empirically, not guessed: the fraction of PASSING attempt-1 rows actually marked `control_sample` (undiluted by the failing population) sits at 0-20% noise before `question_idx~4650` and jumps to 33-100% after - consistent with 8% before, 60% after. Rows now weighted by whichever rate was truly active when generated.

Corrected result:

	Buggy	Corrected
Optimal cutoff (Youden's J)	0.68	**0.70**
Bootstrap median / mode	0.69 / 0.68	0.72 / 0.70
95% bootstrap CI	[0.68, 0.71]	**[0.69, 0.75]**
AUC	0.619	0.629

Conclusion: 0.75 holds up. It sits inside the corrected 95% CI - at the edge rather than dead center (the point estimate shifted a bit lower, ~0.70-0.72, with the full dataset), but not contradicted. Youden's J stays nearly flat across 0.69-0.75 (0.14-0.19) - a broad near-optimal plateau, not one sharp peak, so 0.75 landing slightly off the exact peak isn't a meaningful miscalibration. No threshold change made - the dataset is complete and immutable, so `DEFAULT_THRESHOLD` only affects the live demo or a future run, neither in progress. This closes the threshold -calibration thread that ran from Section 20 through here: three separate sampling/circularity bugs caught across the investigation (Section 20's verifier_judge_agreement circularity, Section 24's initial case-control miscount, this one), each corrected before being reported as fact rather than after - the pattern this whole project has run on since Section 8.

35. Deliverables built: technical report, defense prep, and a blind ground-truth study

With data collection complete (Section 33), turned the raw work into consumable deliverables and set up the one thing the project lacked by necessity - a real ground-truth anchor.

Technical report (`docs/project-report.html`): a structured writeup of the whole project - problem statement, six-stage architecture, the label-free methodology, five key findings (judge selection, the anchoring bug, position/identity bias, the threshold proof, the open Mistral question), limitations stated plainly, contributions, and future work. Distilled from this 35-section log into something a reader/panel consumes top to bottom.

Defense prep (`docs/defense-prep.html`): a 60-second opening pitch, a six-step spoken walkthrough, the numbers worth memorising, and tested answers to eight anticipated questions ordered from foundational to hardest - including how to handle the Mistral question honestly and how to turn the three self-caught measurement bugs into the strongest material rather than hiding them.

Ground-truth study (`ground_truth_study/`): the scope decision (Section 34's future-work fork) was resolved by the user in favour of a small human study to settle the Mistral question. Built the harness for a methodologically sound version rather than having an LLM fake the ground truth (which would reintroduce the exact circularity the project fights): - `build_study.py` samples a BALANCED set (equal Judge-picked-Mistral and Judge-picked-LLaMA3 pairs, 15+15=30 by default) from the 4,909 eligible final rows where the Judge expressed a real preference, BLINDS it (no model names, no Judge verdict in the sheet; answer order independently re-randomized), and writes the identity/verdict mapping to a separate `answer_key.json` the rater never opens. Blinding verified: zero model-identity strings in the rating sheet. - Humans (the user + lab peers as additional raters) fill `rating_sheet.csv` - which answer is more factually correct, plus any outright errors - fully blind. - `analyze_ratings.py` joins verdicts back to the key and runs an exact two-sided binomial test on whether humans side with Mistral, with LLaMA3, or at chance - directly adjudicating the Judge-vs-Verifier disagreement from Sections 29-31. The three outcomes and their interpretations are pre-registered in the script so the conclusion can't be rationalized after seeing the result.

This is the honest closing state: the pipeline and dataset are a complete, defensible FYP on their own; the ground-truth study is the one piece of real accuracy that would let the project make a verified (not just self-consistent) claim, and its harness is ready for human raters.

36. Threshold lowered 0.75 -> 0.72: the bootstrap-median optimum

Revisited the deployed threshold after re-running `training/optimal_threshold_roc.py` on the complete 5000-question dataset (identical result to Section 34): ROC/Youden's-J point optimum **t = 0.70** (AUC 0.629), 2000-resample bootstrap **median 0.72, mode 0.70, 95% CI [0.69, 0.75]**, two-proportion test p = 0.011. The J curve is near-flat across 0.70-0.74 (J ~ 0.17-0.19, with tiny non-monotonic wiggle), so any value in that band is statistically interchangeable.

Section 24/34 had adopted **0.75** as a deliberate *recall-first* choice (sensitivity 86.7% / specificity 30.2% - catch most improvable answers, accept many wasted regenerations). On reflection that sits at the very top edge of the CI, not its centre. Switched `DEFAULT_THRESHOLD` to **0.72** - the bootstrap-median optimum and a balanced middle ground:

t	sensitivity	specificity	Youden J	character
0.70	54.6%	64.4%	0.190	Youden point optimum (balanced)
0.72	**66.7%**	**50.3%**	**0.170**	**bootstrap median - adopted**
0.75	86.7%	30.2%	0.169	former recall-first value

Descriptive effect on the final score distribution: at 0.72, 37.0% of the 5000 final answers fall below the bar (vs 63.3% at 0.75) - mean final confidence is 0.734 regardless (a fixed property of the data). The dataset is complete and immutable, so this only affects the live demo and any future generation run; no data was regenerated. Updated `utils/scoring.py`, restarted `server.py`, and synced the value and its justification through `docs/research-paper.{html,pdf}` and `docs/project-report.{html,pdf}`. (The presentation deck and defense-prep still cite 0.75 and can be synced on request.)

(Log continues below as further tests complete.)

---

37. The Combiner was biased toward its own answers - measured, decomposed, and replaced with gemma2:9b

The Combiner was `llama3`, and `llama3` is also one of the two answer generators. So on every question it was merging its own writing against Mistral's. Section 8 had already recorded the *symptom* of this - `final_answer`-referenced BERTScore/BLEU/ROUGE were systematically biased toward `answer_a` - but the response at the time was to drop those metrics from the confidence score. That removed the bias from the **measurements** and left it in the **shipped answer**. This section finally measures the thing itself.

Tier 0: measuring it on the existing corpus (no regeneration)

`training/combiner_bias_analysis.py`, all 5000 final rows. For each row, embed `answer_a`, `answer_b` and `final_answer` with the same all-MiniLM-L6-v2 encoder the pipeline already scores with, and ask which source answer the final answer sits closer to:

```
delta_position = cos(final, answer_a) - cos(final, answer_b)
lean_llama     = +delta_position if model_a == llama3 else -delta_position
```

The headline number is **58.7% of finals closer to llama3 (p=3.5e-35)**, but that figure is not interpretable on its own, because the combiner turns out to have a *second* bias. Crossing position against identity separates them:

Effect	Size	Significance
Identity - llama3 advantage, position held fixed	**+18.1 pp**	p ~ 1.7e-89
Position - slot-1 advantage, averaged over identity	**+10.8 pp**	p ~ 5.7e-53
Length confound	r = +0.029	ruled out

Length was a real candidate and is genuinely out: llama3 writes longer answers (366 vs 317 words), but the correlation between length advantage and lean is only +0.029, and the lean *persists at 61.1%* among length-matched pairs (word counts within 10%, n=1645, p=2e-19). The combiner is not simply keeping more of the longer answer.

The mechanism is selective obedience, not style recognition

The identity effect is entirely conditional on the judge agreeing with it:

Judge picked	llama3-lean in the final answer
llama3's answer	**69.4%** (p ~ 5e-74)
mistral's answer	**50.7%** (p = 0.465, **not significant**)

The combiner followed the judge 69.4% of the time when the judge pointed at llama3, but only 49.3% when it pointed at Mistral - a **20 pp obedience gap**. It amplified the judge when the judge favoured its own answer and ignored the judge when it did not. The full 2x2x2 makes it stark; every cell is significant:

Slot 1 holds	Judge picked	n	leans to slot 1
llama3	llama3	620	87.7%
llama3	mistral	1815	**64.0%**
mistral	llama3	1530	38.0%
mistral	mistral	1035	72.7%

Row 2 is the finding: the judge picked Mistral, Mistral's answer was sitting right there in slot 2, and the final answer *still* landed closer to llama3 64% of the time.

A real ordering bug in the live server, found while fixing this

`server.py` kept `answer1` as llama3 always (only the Verifier/Judge *view* was randomized, with parsed scores remapped back via `remap_ab`). But `verification` and `judgement` are the **raw agent text**, written against the randomized view, and the judge's Overall Reasoning names "Answer 1"/"Answer 2" explicitly in that order. The combiner was therefore handed UI-order answers alongside judge-order feedback: whenever `swap` was true - **half of all live queries** - it was told "Answer 1 is more relevant" about the answer in the *other* slot.

The batch generator never had this bug: it builds the combiner prompt from `answer_a` / `answer_b` and the raws produced against that same order. Live only.

Fixed by passing the combiner the judge-view order, which also stops the +10.8 pp position pull from parking on llama3 on every single query.

The change: gemma2:9b as Combiner

`gemma2:9b` is neither generator, nor the Verifier (phi3), nor the Judge (qwen3.5:9b), so it has no answer of its own among the two it merges - the identity term is removed **by construction** rather than reduced. It is a different model family, already local (no download), and in the replay pilot it respected the prompt's 150-word cap (mean 128, max 151 words) where llama3 systematically overshot it (mean 237, max **407**). That cap had effectively not been in force.

This is safe with respect to every published result. The combiner is strictly terminal: `compute_weighted_score` never takes `final_answer` as a signal, and the combiner runs once *after* the regeneration loop closes. The 5000-question dataset, the 0.72 threshold calibration and the judge-selection results are all untouched. End-to-end smoke test after the swap: all six stages ran, final answer 136 words.

What this does NOT fix: the +10.8 pp position effect. That is a property of how a transformer reads a prompt, not of model identity, so changing the combiner model cannot address it, and the per-question coin flip only makes it unbiased *with respect to model identity* rather than absent. Roughly 61% of final answers are still dominated by one slot rather than genuinely merged.

Open / next

- **Tier 1 replay** (`training/combiner_replay.py`) is running: 400 questions x {llama3, gemma2:9b} x {forward, swapped order}, every other input frozen, which isolates identity lean (orders averaged, position cancels) from position lean (orders compared). The harness remaps the A/B labels inside `verifier_raw`/`judge_raw` when it swaps - without that it would feed the combiner a verdict pointing at the wrong text and contaminate the effect being measured. This is the same bug class as the server one above.
- **Extractive MMR combiner** proposed as a third arm: bias-free by construction, deterministic, faithful by construction, exact length control, and it makes the source split directly countable instead of inferred. Known failure mode: MMR's diversity term *rewards* selecting contradictions, since a contradiction is maximally non-redundant - which conflicts with the combiner's job of removing them. Hybrid (MMR selects content, a small LLM only smooths surface) is the likely production design.
- A useful by-product: for an MMR arm the true source split is known by construction, so it calibrates the embedding-lean metric used throughout this section.